David King
Dr. Tong Zhang
Advanced Computer Systems
23 October 2026

Classwide Project: SSD Performance Profiling

Introduction

This report looks at how request size, access pattern, read/write balance, queue depth, and file range affect SSD throughput and latency. We used FIO to test a KIOXIA NVMe SSD with temporary files and direct I/O. Throughput mostly leveled off at QD 32, while running more requests at once increased tail latency. Repeated reads also looked much faster when they used the host page cache.

System Configuration

Item	Configuration
SSD / capacity	KIOXIA KXG8AZNV1T02 LA; 953.9 GiB reported by Linux
Interface	NVMe; XG8 specification: PCIe 4.0 x4, NVMe 1.4
Storage target	Disposable regular files on ext4 (/dev/nvme0n1p4)
CPU	Intel Core i7-13800H; one FIO job pinned to CPU 0
Operating system	Ubuntu Linux; kernel 6.8.0-147-generic, x86-64
Tool/engine	FIO 3.36; Linux asynchronous I/O (libaio)
Frequency / SMT	powersave governor; SMT enabled
Test ranges	1 GiB main file; 16 GiB file for range and burst follow-ups

Measurement Procedure

1. Create a new temporary file, initialize the whole file, and check 16 MiB with CRC32C before timing.
2. Use aligned requests and direct I/O, with one job pinned to CPU 0. Generate pseudorandom write buffers.
3. For the original set of tests, run five 10-second trials for each configuration in shuffled order. Initialize the file before measuring, and read the data once before the buffered comparison. The newer block-size and mix tests measure three seconds after one second of warmup. The range tests measure six seconds after two seconds of warmup.
4. Report the median and interquartile range (IQR) across trials. Latency is the total I/O latency reported by FIO, including submission and completion time. For mixed workloads, weight the read and write averages by the number of completed requests. Tail-latency values summarize the percentiles from each trial. Burst curves use arithmetic averages across five runs.

Pre-experiment Expectations

We expected larger requests to move more bytes per second but take longer to finish. We also expected higher queue depth to raise IOPS until extra requests mostly added waiting time. Read/write mixes would likely behave differently from a simple blend of the read-only and write-only results. Repeated buffered reads would likely benefit from the host page cache.

Experiments and Results

Unloaded Latency Baselines

At QD 1, we tested 4 KiB random reads and writes and 128 KiB sequential reads and writes over the 1 GiB file. Table 1 reports the median, mean, p95, and p99 latency across five trials.

Workload	Mean (µs)	p95 (µs)	p99 (µs)
4 KiB random read	49.72	51.97	55.55
4 KiB random write	11.02	13.89	16.77
128 KiB sequential read	125.98	156.67	183.30
128 KiB sequential write	48.20	74.24	84.48

Table 1. QD 1 total I/O latency. Values are medians across five 10-second trials.

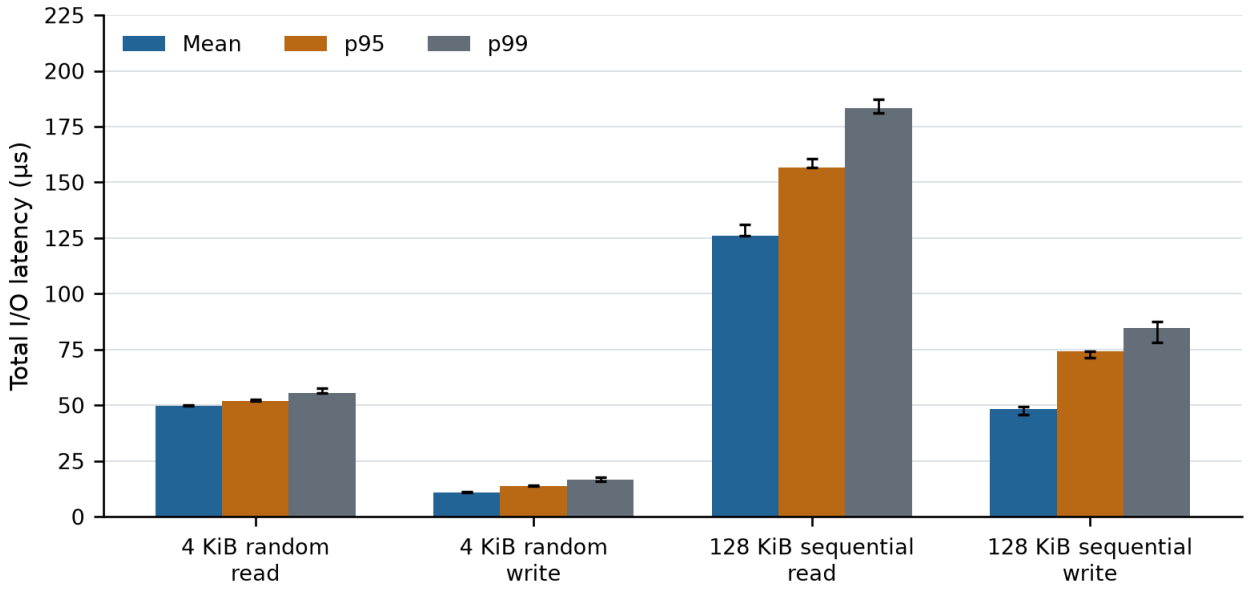


Fig. 1. QD 1 latency for four baseline workloads over 1 GiB. Bars show medians; error bars show trial IQRs. Lower is faster.

Writes finished faster than reads in both workload pairs: 11.02 versus 49.72 µs for random I/O and 48.20 versus 125.98 µs for sequential I/O. Sequential writes also had a wider gap between mean and p99 latency, reaching 84.48 µs at p99. Buffering in the controller may help explain the short write-completion times.

Block Size and Access Pattern

We compared sequential and random reads across 14 block sizes (4 to 256 KiB), keeping QD at 16 and the file range at 1 GiB. For each size, we ran five paired trials. We alternated which access pattern ran first and shuffled the block-size order. We measured IOPS, bandwidth, and latency in the same trials.

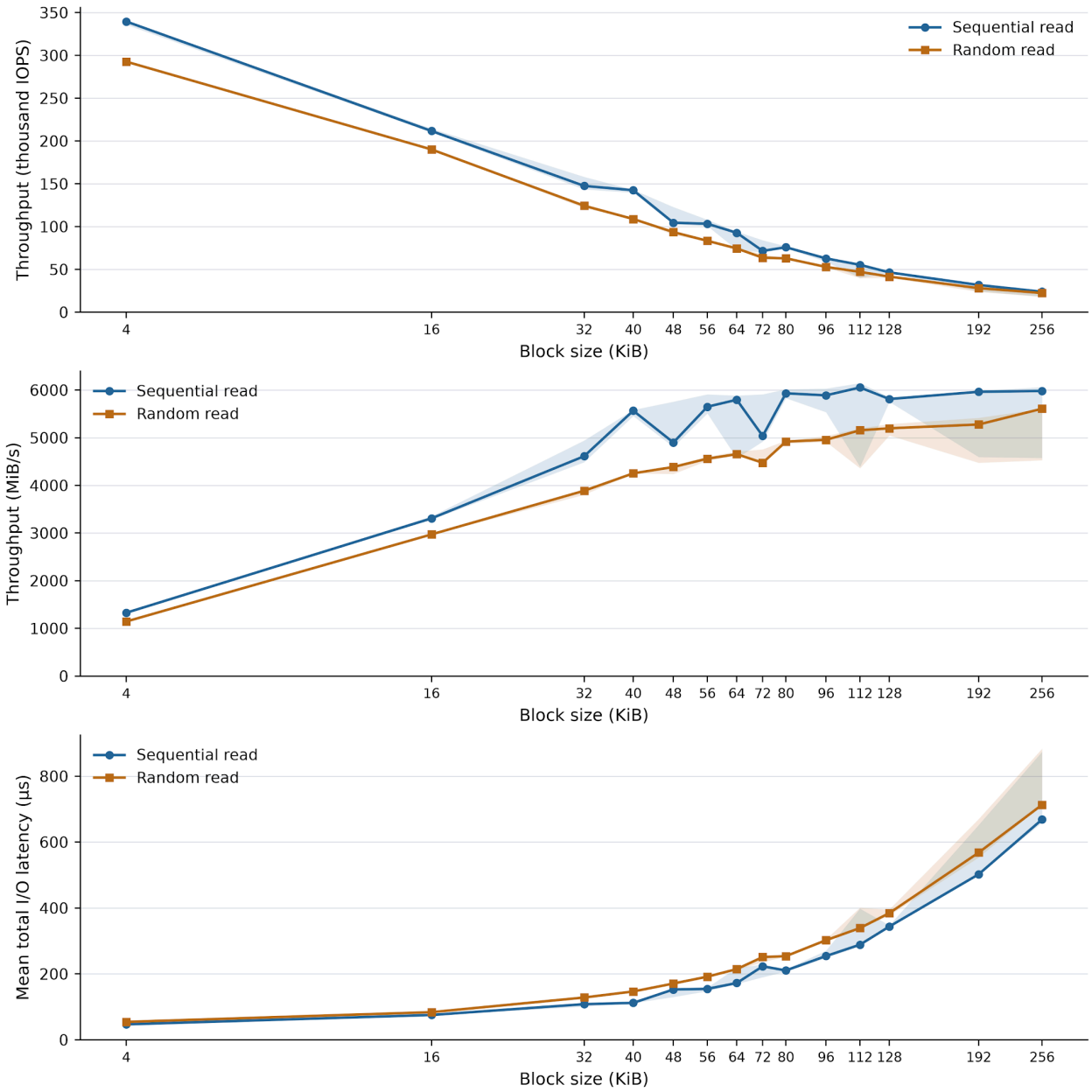


Fig. 2. Read throughput and latency versus block size at QD 16 over 1 GiB. Lines show medians of five paired trials; bands show IQRs.

Sequential bandwidth rose from 1,326.58 MiB/s at 4 KiB to a peak of 6,053.83 MiB/s at 112 KiB, then stayed near 6,000 MiB/s through 256 KiB. Over the same range, sequential IOPS fell from 339,600 to 23,918 and mean latency rose from 46.93 to 668.58 µs. Larger transfers spread the overhead of each request across more bytes, but each request takes longer to finish.

Sequential reads had higher median bandwidth at every size we tested. At 64 KiB, they reached 5,796.38 MiB/s, compared with 4,654.72 MiB/s for random reads. At 256 KiB, the gap narrowed to 5,980.67 versus 5,608.38 MiB/s. The sequential peak is about 6,348 MB/s, or 91% of the XG8's advertised 7,000 MB/s maximum [1]. Bandwidth leveling off across several sizes suggests a limit in the controller, flash, or connection to the host.

Testing many block sizes also showed a few exceptions to the overall trend. Sequential bandwidth dips at 48 and 72 KiB, and results vary more at several larger sizes. These details matter when choosing a request size. At around 64–128 KiB, sequential reads are already close to the highest bandwidth we observed. Moving from 128 KiB to 256 KiB roughly doubles latency, so the larger blocks bring only a small bandwidth gain at the cost of longer requests.

Read/Write Mix

We tested read/write mixes using 4 KiB random I/O at QD 16 over 1 GiB. We increased the share of writes from 0% to 100% in 10-percentage-point steps and ran five trials per mix in shuffled order. Table 2 shows the four ratios required by the project spec.

Read/write (%)	Thousand IOPS	Mean latency (µs)
100/0	295.33	53.95
70/30	290.12	54.56
50/50	288.85	54.56
0/100	241.01	64.83

Table 2. Required read/write ratios from the five-trial mix sweep; values are medians.

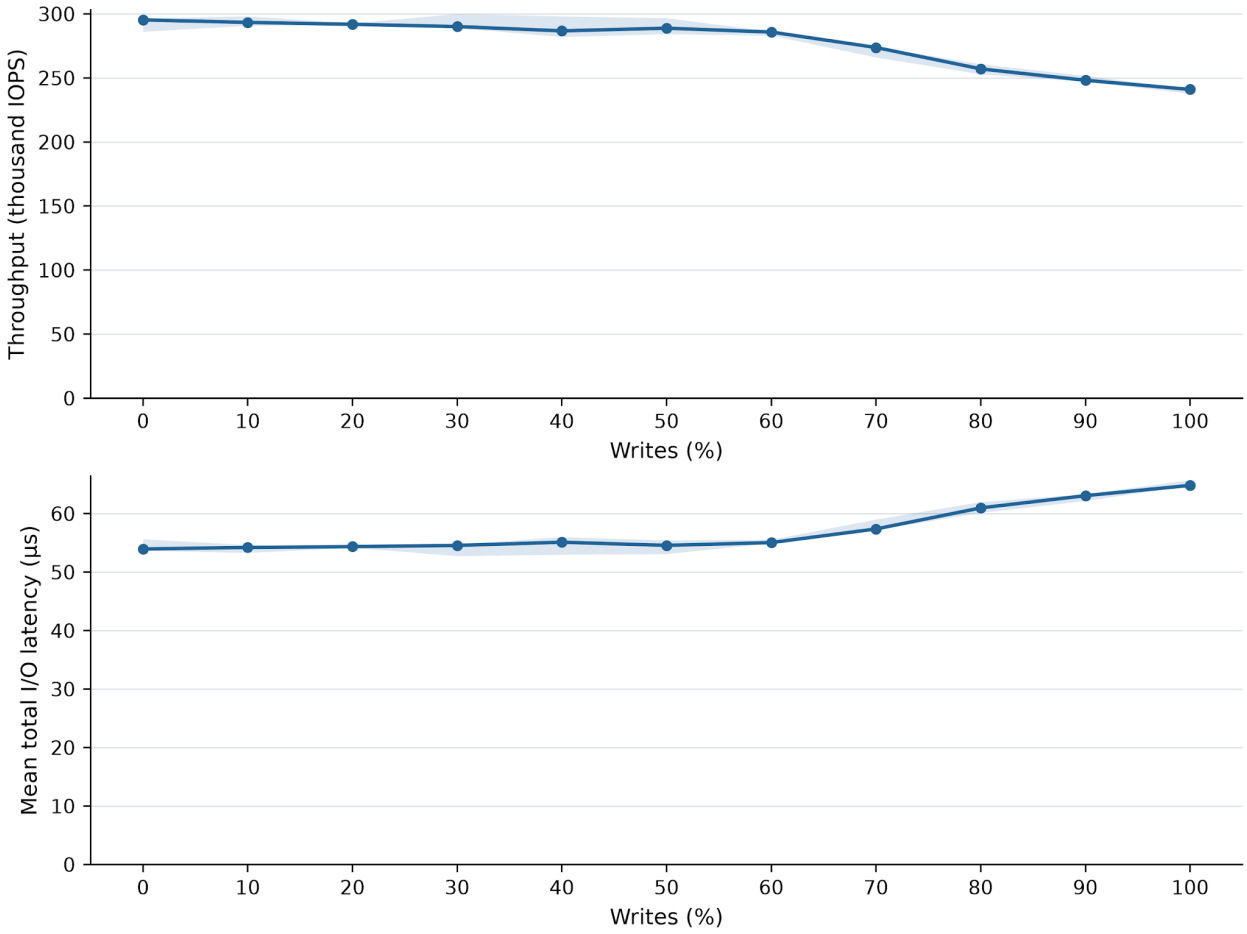


Fig. 3. Throughput and mean latency versus write percentage; 4 KiB random I/O, QD 16, 1 GiB. Bands show trial IQRs.

Throughput stayed between about 286,000 and 295,000 IOPS up to 60% writes, with mean latency near 54–55 µs. Above 60% writes, throughput fell toward 241,012 IOPS and latency rose to 64.83 µs. The write-only test produced 18.4% fewer IOPS than the read-only test. Sustained writes require more flash programming and internal data management, which may explain why write-heavy workloads were slower.

Queue Depth and Concurrency

We tested QD values from 1 to 256 with 4 KiB random reads over 1 GiB. Throughput rose from 20,622 IOPS at QD 1 to 283,768 at QD 16 and 377,372 at QD 32. Mean latency rose from 48.27 to 56.13 and 84.56 µs at those same depths. Running requests at the same time lets the controller keep working while earlier requests finish.

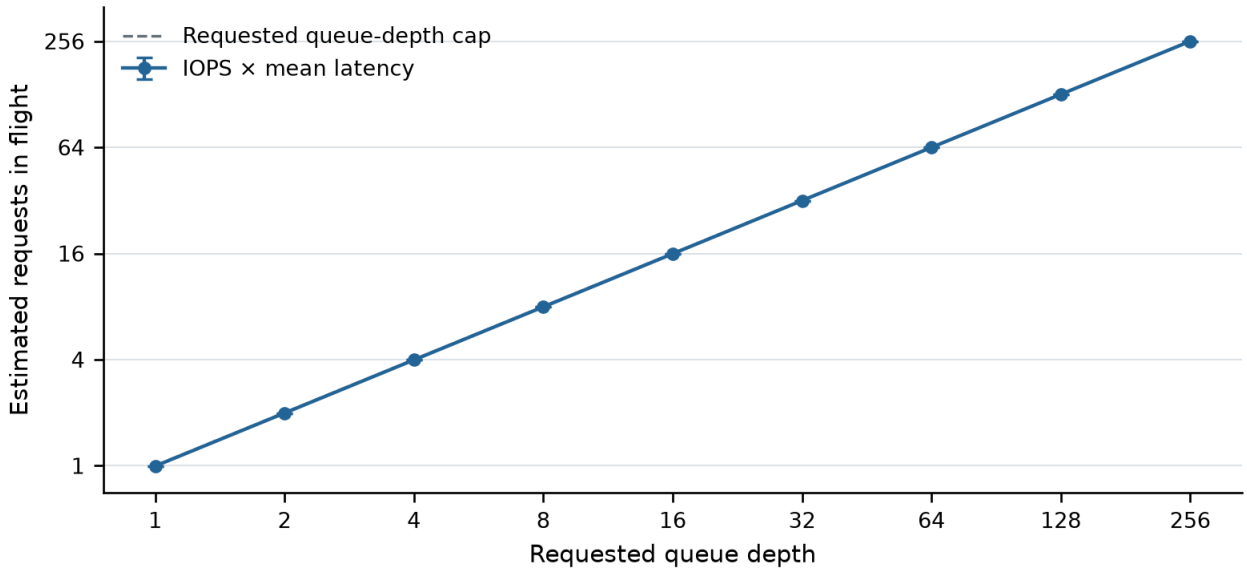


Fig. 4a. Average requests in flight calculated per trial as IOPS × mean latency in seconds, compared with requested QD. Error bars show IQRs.

Little's Law connects the average number of requests in flight L, throughput λ, and mean latency W: L = λW. At QD 32, the calculated median was about 31.91 requests in flight. FIO's recorded queue depths also show that the workload kept the queue busy. Figure 4a shows how closely these values match across the tested depths.

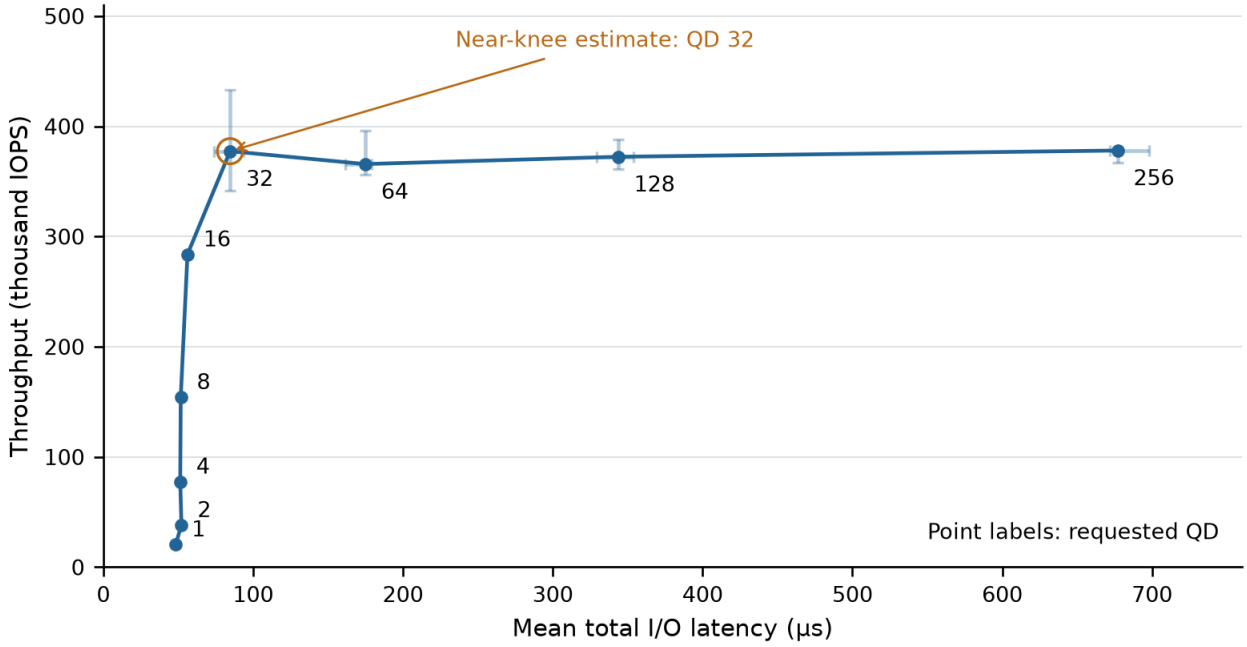


Fig. 4b. Throughput versus mean latency for 4 KiB random reads over 1 GiB. Labels identify QD; bars show IQRs on both axes. The marked knee uses the 90%-of-peak criterion.

QD 32 was the first tested depth to reach at least 90% of peak median throughput. From QD 32 to QD 256, throughput rose by only 0.16%, to 377,984 IOPS, while mean latency rose eightfold to 676.90 µs. This makes QD 32 a practical stopping point for increasing queue depth, shown as the throughput knee in Figure 4b. Once throughput levels off, Little's Law predicts that extra requests mainly add waiting time.

Tail Latency

We compared the QD 8 and QD 32 random-read trials at p50, p95, p99, and p99.9. QD 8 keeps a moderate number of requests active, while QD 32 is where throughput starts to level off in our queue-depth tests.

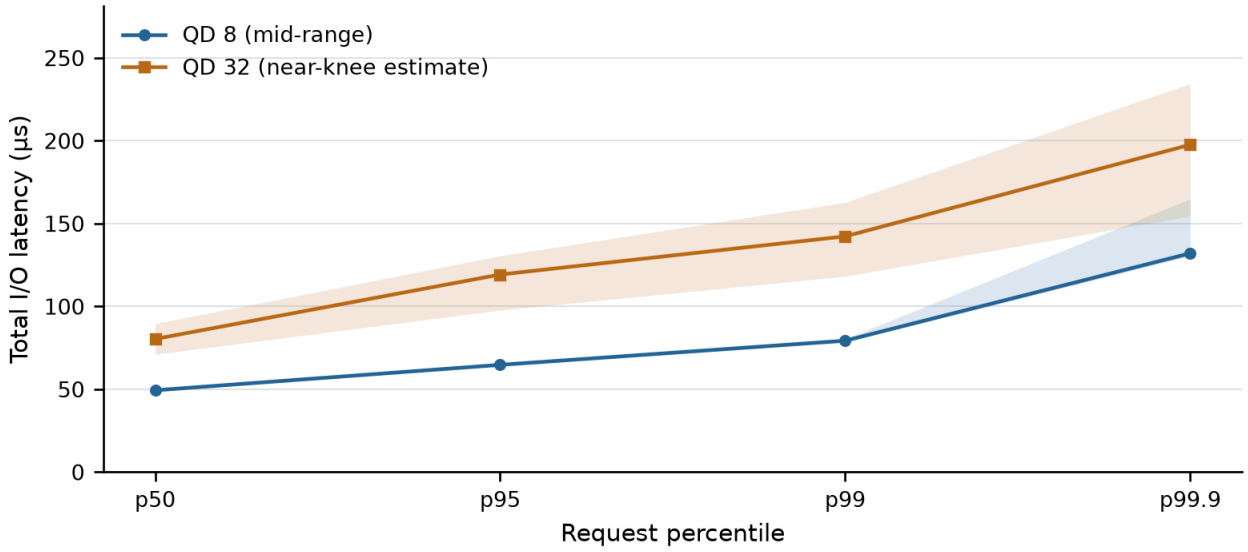


Fig. 5. Total I/O latency percentiles for 4 KiB random reads at QD 8 and QD 32. Points show medians of five per-trial percentiles; bands show IQRs.

At QD 8, p50, p95, p99, and p99.9 were 49.41, 64.77, 79.36, and 132.10 µs. At QD 32, they rose to 80.38, 119.30, 142.34, and 197.63 µs. Throughput grew by about 2.45 times between these depths, while p99 rose 79%. Results also varied more between trials at QD 32. So QD 8 gives faster, more consistent responses, while QD 32 gives higher throughput.

File Range

We tested eleven file ranges from 16 MiB to 16 GiB, doubling the range at each step. We ran eight rounds in shuffled order using the same initialized 16 GiB file, 4 KiB random reads, direct I/O, and QD 16.

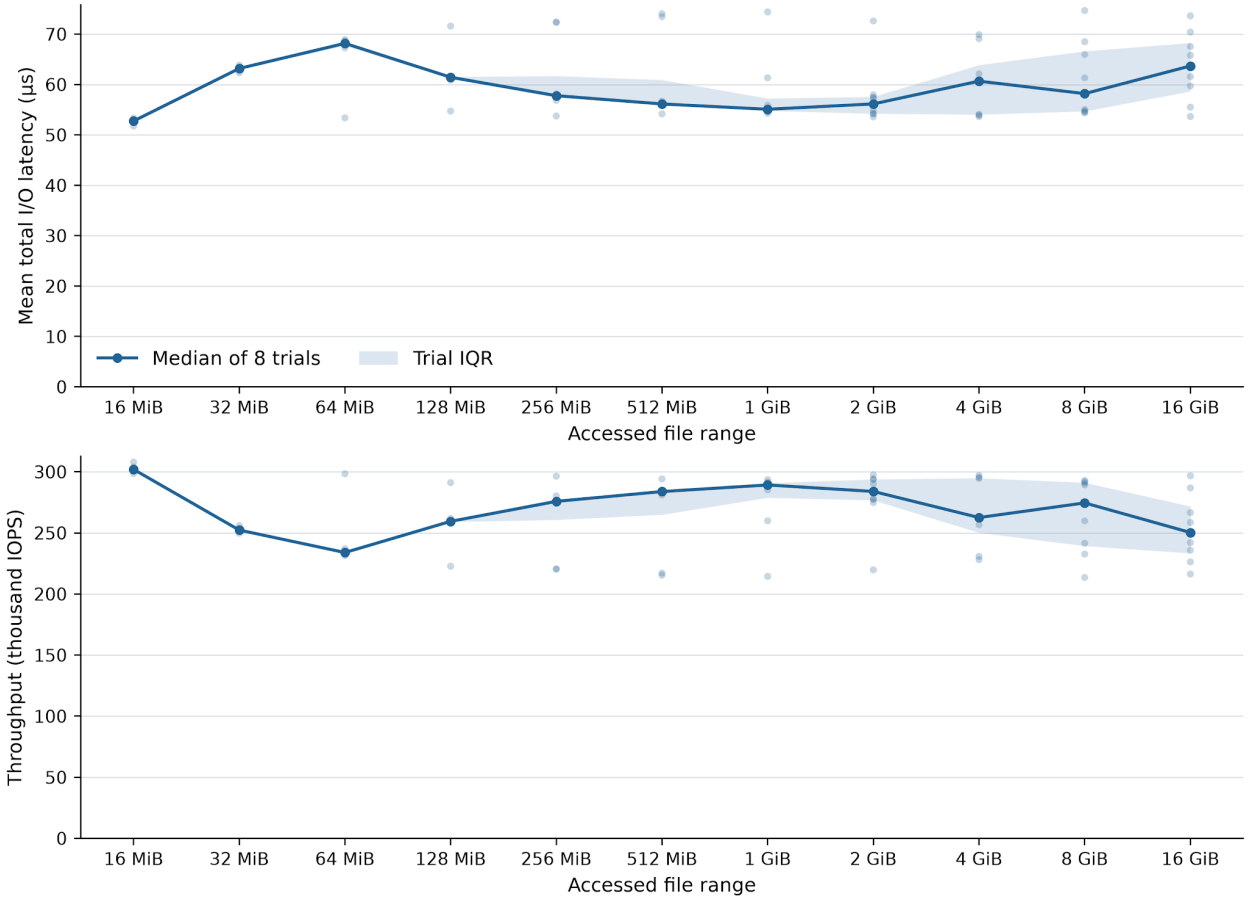


Fig. 6. Random-read latency and throughput versus file range; 4 KiB requests, QD 16. Lines show eight-trial medians, bands show IQRs, and faint points show individual trials.

Latency rose from 52.75 µs at 16 MiB to 68.16 µs at 64 MiB, then fell to 55.08 µs at 1 GiB. Throughput recovered from 233,935 IOPS at 64 MiB to 289,242 at 1 GiB. At 16 GiB, median throughput was 250,324 IOPS, and results varied more between trials. Performance fluctuated as the range grew. This suggests that address locality, how the controller maps data, and changes in device activity may all play a role. A larger file range did not always mean slower performance.

Sequential Write Burst

We ran five 60-second tests that repeatedly overwrote the same initialized 16 GiB file using 128 KiB sequential writes, direct I/O, and QD 16. Before each run, we left the drive idle for 30 seconds. Figure 7 averages the matching 250 ms windows from all five runs through 59.75 seconds, giving each run equal weight.

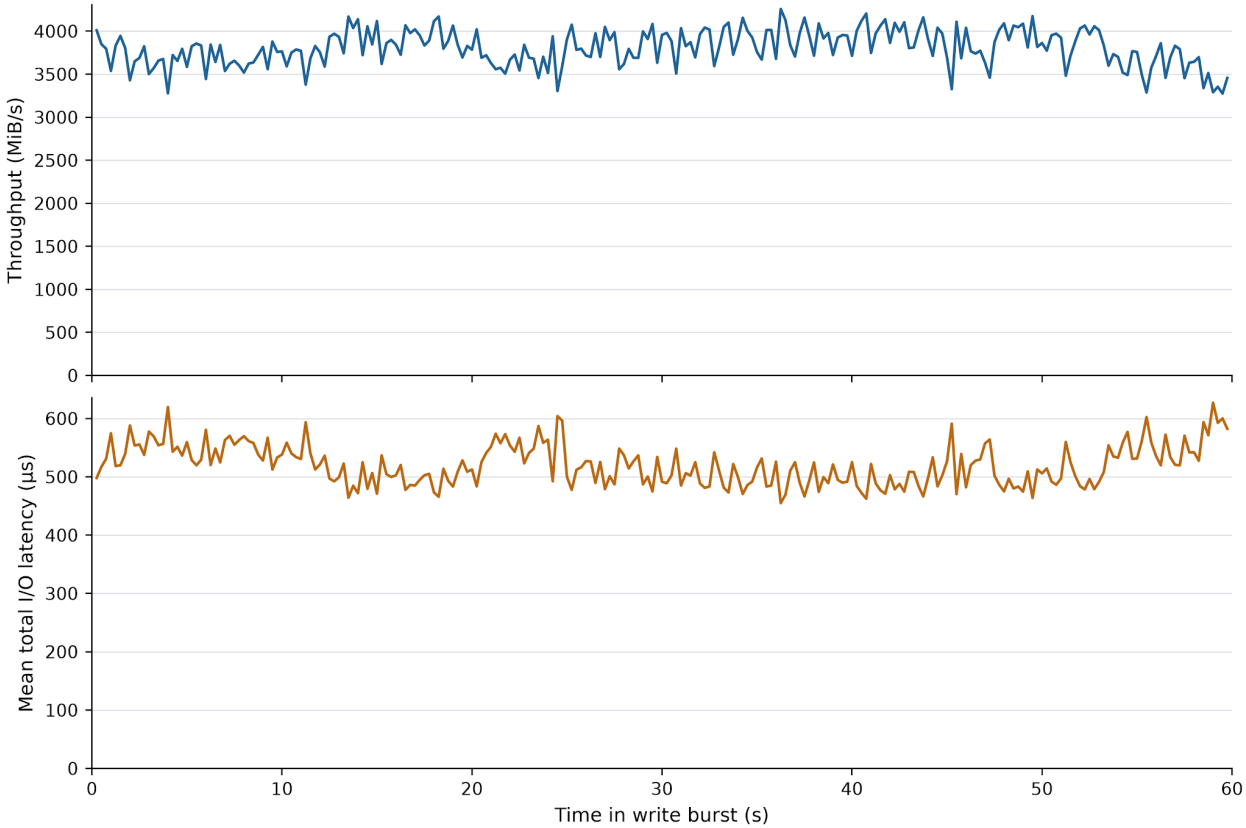


Fig. 7. Mean throughput and latency across five 60-second write bursts; 128 KiB sequential writes, QD 16, 16 GiB. Each plotted window averages five runs.

Average throughput moved between about 3,275 and 4,257 MiB/s, while mean latency ranged from 455 to 627 µs. Throughput recovered after earlier slow periods and dropped again near the end. The average throughput for each full run ranged from 3,510 to 3,968 MiB/s. When throughput dipped, latency rose, which fits with temporary slowdowns in handling writes. Cache management and background flash work may help explain these changes.

Benchmark Pitfall: Page Cache

We compared buffered and direct 128 KiB sequential reads at QD 1, repeatedly reading the same 16 MiB range. For the buffered test, we read the range once before timing so later reads could reuse the data in memory.

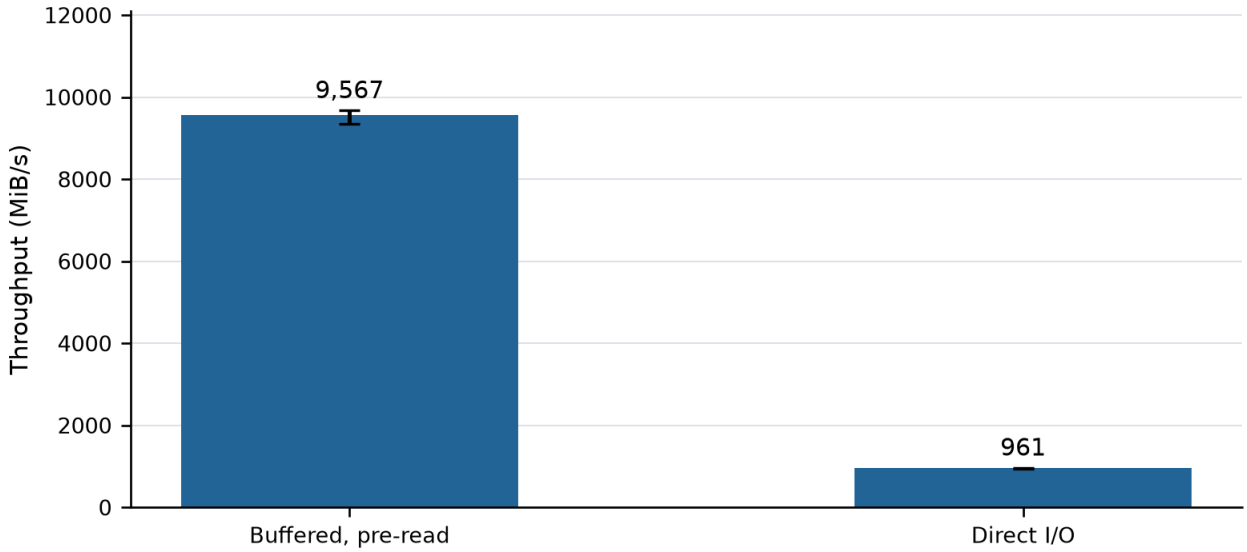


Fig. 8. Repeated 128 KiB reads over 16 MiB at QD 1, comparing buffered and direct I/O. Bars show five-trial medians; error bars show IQRs.

Buffered reads reached 9,566.83 MiB/s, compared with 961.40 MiB/s for direct reads. That makes the buffered median 9.95 times higher. The repeated range easily fits in system memory, so the host page cache can serve later requests. Calling this SSD bandwidth would make the drive look much faster than it really is. Direct I/O is the right comparison for the other experiments.

Validation and Reproducibility

The CRC32C write/read checks passed, and file initialization finished successfully. All 175 timed observations in the original tests returned FIO error=0. We also checked 140 selected block-size observations, 55 mix observations, 88 range observations, and five write bursts from the follow-up tests. We confirmed the settings, checked that all expected trials were present, verified the byte-count calculations, and checked the saved-data hashes. For the bursts, we also checked the log timing and matched time windows.

The repository includes FIO configurations, scripts, raw JSON, processed CSVs, metadata, and figure sources. From project3-ssd, regenerate the original analysis with make plots RUN_DIR=output/20261022T215132.687552Z. Regenerate each follow-up with ../.venv-analysis/bin/python scripts/<study>_repeat.py --plot <saved-run-directory>, using block, mix, range, or burst for <study>; the block command also requires --first-trials 5. report/source-manifest.json identifies the exact datasets, images, and hashes used here.

Each figure summarizes its own set of tests. We kept the original and follow-up results separate because they used different durations, file sizes, and prior device activity.

Measurement Limits

These results come from one mounted system SSD running under the powersave governor. Background activity and earlier drive use may have affected the results. We tested only part of the drive, and the filesystem and SSD decide where data is physically stored. Write latency measures when FIO receives confirmation that a write has completed; the timed tests use ordinary asynchronous writes. The 60-second bursts show only short-term behavior. We saved temperature readings and their hardware-monitor paths, but we would need to identify the SSD sensors before linking slowdowns to thermal throttling. Trial IQRs show the middle 50% of observations. Averaging the burst runs smooths out brief changes that happen at different times in each run.

AI Use

Codex helped with FIO configurations, data collection and analysis scripts, plots, and drafting. We verified the generated work using CRC32C checks, setting and calculation checks against raw FIO output, saved-data hashes, and comparisons between plotted values and source tables. Follow-up tests refined the original block-size, mix, range, and burst measurements.

References

[1] KIOXIA, XG8 Client SSD Product Brief, Rev. 1.2 (2024). Interface and advertised maximum sequential-read speed.